

中文题目： 基于时空信息的旅游景点推荐系统的设计与实现

外文题目： Design and Implementation of Tourist Attraction
Recommendation System Based on Spatiotemporal
Information

摘 要

在互联网内容爆炸的时代，大规模的知识和信息迅速充斥进人们生活的方方面面，想要获取直接对个人有价值的内容变得愈加困难。而旅游产业作为国家 GDP 的支柱产业之一，也是人们日常生活之余最热门的娱乐方式。对于传统旅游系统来说，如何在海量旅游景点中提供给用户最合适的，即用户最有可能选择的个性化景点成为一个重要问题。

基于以上问题，本文设计并实现了基于时空信息的旅游景点推荐系统，期望根据用户的时空信息，直接向用户推荐用户最可能感兴趣的旅游景点。本文针对传统旅游系统推送的低有效性，实现了基于用户时空信息分类的协同过滤推荐算法。系统获取用户对某景点的评分，再根据用户注册时填写的地理位置信息，将地理位置相同的用户分组，对同组用户的评分数据使用基于用户的协同过滤推荐算法进行计算，使用皮尔逊相似度计算近邻用户，训练模型输出与当前用户最相似的景点信息，向用户推荐与其喜好与地理位置信息都更加匹配的景点集合。系统基于 Java 的 Springboot 框架和 Vue.js 框架，搭建前后端分离 Web 应用，用户端分为首页推荐、景点搜索、景点收藏评分、用户注册登录、用户信息管理模块，管理员端分为收藏统计图、景点管理、用户管理模块，推荐模块使用基于 Hadoop 的 Mahout 推荐算法框架实现。经过实验测试，本旅游推荐系统能够正常运行并实现设计的功能。

关键词：旅游景点；时空信息；推荐系统；协同过滤； Mahout 框架；

Abstract

In the era of Internet content explosion, large-scale knowledge and information are rapidly flooding into all aspects of people's lives. It is increasingly difficult to obtain content which is directly valuable to individuals. As one of the pillar industries of the country's GDP, the tourism industry is also the most popular form of entertainment for people in their daily lives. For traditional tourism systems, how to provide users with the most suitable and personalized attractions that are most likely to be chosen among a large number of tourist attractions has become an important issue.

Based on the above problem, this article designs and implements a tourism attractions recommendation system based on spatial-temporal information, expecting to directly recommend the tourist attractions that users are most likely interested in based on their spatial-temporal information. This article addresses the low effectiveness of traditional tourism system recommendations by implementing a collaborative filtering recommendation algorithm based on user spatial-temporal information classification. The system obtains user ratings for a certain attraction, and then groups users with the same geographical location according to the information filled in when the user registers. The rating data of users in the same group is calculated using a user-based collaborative filtering recommendation algorithm, and Pearson similarity is used to calculate the nearest neighbors. The training model outputs the most similar attraction information to the current user, and recommends attractions that are compatible with both preference and geographical location information to the user. The system is built with Springboot framework based on Java and Vue.js framework, creating a front-end and back-end separation Web application. The client side has home page recommendation, attraction search, attraction collection rating, user registration login, and user information management modules, while the administrator side is divided into collection statistics chart, attraction management, and user management modules. The recommendation module uses the Mahout recommendation algorithm framework based on Hadoop to achieve. Through experimental testing, this tourism recommendation system can operate normally and achieve the designed functions.

Key Words: tourism attractions; spatial-temporal information; recommendation system; collaborative filtering; Mahout framework;

目 录

摘要	I
Abstract	II
目录	III
第一章 引言	1
第一节 研究背景及意义	1
第二节 国内外研究现状	1
第三节 文章的组织结构	3
第二章 系统相关技术	4
第一节 相关技术介绍	4
2.1.1 前后端分离架构模式	4
2.1.2 协同过滤推荐算法	5
2.1.3 Mahout 推荐系统架构	7
第二节 本章小节	8
第三章 系统需求	9
第一节 设计目标	9
3.1.1 用户需求分析	9
3.1.2 管理员需求分析	10
第二节 本章小节	10
第四章 系统设计	11
第一节 系统模块设计	11
4.1.1 信息收集和整合模块	11
4.1.2 信息保存及展示模块	12
4.1.3 功能权限校验机制设计	13
第二节 推荐模块设计	14
4.2.1 构建用户评分矩阵	14
4.2.2 相似度计算	15
4.2.3 生成推荐	16
第三节 数据库设计	17
4.3.1 用户信息表	18

4.3.2	景点信息表	18
4.3.3	景点收藏表	18
4.3.4	景点评分表	19
4.3.5	用户-景点分数视图表	19
4.3.6	推荐景点结果表	21
第四节	本章小节	21
第五章	系统实现	22
第一节	系统实现方案	22
5.1.1	功能权限校验机制的实现	22
5.1.2	推荐功能的实现	24
5.1.3	信息保存和展示功能的实现	26
第二节	系统实现结果	27
5.2.1	用户注册及登录页面	27
5.2.2	系统首页	27
5.2.3	系统后台管理页面	28
第三节	本章小节	29
第六章	文章总结	30
参考文献		31
致 谢		XXXII

第一章 引言

第一节 研究背景及意义

随着新型冠状病毒疫情的结束，在疫情中最受冲击的产业之一——旅游产业消费加速回暖。据中国互联网络信息中心发布的数据，至 2023 年12 月，在线旅行预订用户规模已达 5.09 亿，较去年增长约20.4%。旅游业对于国家经济增长、地方脱贫、就业促进以及多元产业发展具有重大意义。对于个人来说，旅游是人们不可或缺的娱乐休闲途径之一。随着电子商务的蓬勃发展，旅游行业也积极融入市场，大量旅游景点信息被展示在互联网上，用户能够查询并挑选各类旅游景点，同时参考其他游客的评价做出更明智的选择。

尽管传统旅游景点系统为用户提供了便捷的查询方式，但并未实现真正满足用户个性化的需求的直接推荐方式。对于用户来说，从海量内容里获取到自己感兴趣的景点信息代价过大。对于旅游企业来说，挖掘景点信息中的个性化元素，实现精准推荐成为亟待解决的问题。因此，个性化推荐系统应运而生，它有助于用户快速、高效地获取到感兴趣的景点信息，提升信息到价值转化效率。

而在多样的个性化指标当中，用户的时空信息常被忽略。根据各大旅游平台的统计报告显示，假期内本地、周边旅游订单占比达到 65%，表明用户所在位置与旅游景点的选择具有强关联性。因此，文章期望设计并实现一个基于用户时空信息的旅游推荐系统，该系统可向用户推荐个性化旅游景点，用户可以对景点进行评分、收藏、查询、查看详细信息，管理员端可对景点信息和用户信息进行管理，系统后台执行算法的训练与推荐，投放至用户端交互。

第二节 国内外研究现状

1997 年，Resnick 和 Varian 为电子商务推荐系统下了正式的定义：通过电子商务网站所提供的商品信息及建议来协助用户做出消费的决策^[1]。这种新型的购物方式，使用户能够更直接地获取有效信息，节省大量人工成本，实现自由交易，摆脱时空限制，这种跨越时间和空间的便利为用户带来前所未有的体验。推荐系统的优势显著，可以概括为以下三点：将用户转化为买家；为用户提供个性化服务；提高用户对网站的忠诚度。

亚马逊率先认识到推荐算法在电子商务中的潜力，并成功将其引入，为电子商务发展开辟新道路，对电子商务的发展产生了巨大影响。在国外，亚马逊

继续运用推荐系统，并且随着技术的不断成熟和用户体验要求的不断提高。为了解决冷启动问题，机器学习在训练推荐模型中的应用已经逐渐普及。2006 年，网飞公司设立奖项，表彰那些能够将推荐准确率提高 10% 的杰出学者。随后高精度算法推荐系统开始崭露头角，但它仍然不能完全正确地满足推荐；因此，提高推荐的准确性是衡量推荐算法是否有效的关键，这也是推荐系统发展过程中需要克服的一个难题。

2013 年，Netflix 在官方博客分享了其经典的推荐系统架构。Netflix 推荐架构分为三层：ONLINE 在线层、NEARLINE 近线层、OFFLINE 离线层。三层相互配合又相互独立，确保推荐系统的顺利运行。Embedding 技术也在近期流行起来，它可以从人人推荐、物物推荐、人物推荐中猜测到用户偏好的商品以及可能认识的用户等，在多方面应用都有不错的效果反馈。

尽管国内推荐系统起步晚于国外，但近年来发展迅速。自 2003 年电商元年始，各大电商巨头纷纷将推荐系统引入了线上电子商务中，随着国内互联网的逐渐成熟，这些推荐系统已经能带给人们很好的使用体验。如今国内推荐系统的研究大多集中在协同过滤算法的方向，建立了基于模型的协同过滤体系。李启旭，王巧，郁雪等学者发表了以 LFM 建立模型为基础的相关文献以推动国内推荐系统的建设，也有学者使用 FPFM 推荐框架进行研究。2008 年，淘宝团队推出“i 淘宝”电商推荐模块，这便是淘宝电商推荐系统的前身，三年后百度也加入了行列，提出“一人一世界”的口号，着重于研究个性化推荐的搜索技术。

旅游电子商务是网络信息时代的新业务，近年来已成为旅游研究的热点。国外对旅游消费者行为的研究涵盖在线信息搜索、旅游决策规划、消费者特征感知、满意度评估等。鉴于大部分用户查询旅游信息都是通过网络的方式，旅游系统上消费者对景点的评价成为了重要参考，因其是旅游作为体验性产品的重点价值评估标准之一。Stephen 等学者针对用户是否信任网络上的信息进行了调查与研究^[2]，结果显示用户在网络上搜索到的大部分旅行信息都来自于旅游产品的销售者、专家或其他游客的评价，而这些信息中，用户更在乎其他游客的评价信息，且对其更加信任。这是因为在虚拟网络上，用户普遍认为其他用户的评价能反映产品体验过程的真实情况。

中国的旅游电子商务起步较晚，尽管近十年来中国旅游电子商务的研究取得了很大进展，但仍处于探索和发展阶段。2001 年以前，对旅游电子商务的研究相对缺乏；2001-2006 年，旅游电子商务研究处于萌芽阶段；2007 年被称为在线旅游的元年。2007 年以来，在线旅游与传统产业的结合互补进入稳步发展期。

然而，大多数旅游电子商务网站都专注于网站功能开发和运营模式的研究，对网站推荐系统的研究很少，景点评价、用户喜好等信息对网站本身的深入发展和把握用户使用系统的个性化偏好具有重要意义。

第三节 文章的组织结构

文章介绍了一个基于时空信息的旅游推荐系统的设计与实现，该系统使用基于用户的协同过滤算法向用户推荐个性化景点信息。文章分为六章节，各章节内容如下：

第一章介绍了系统的设计背景及意义和国内外研究现状。首先简单介绍了近年来旅游产业的现状和旅游系统的问题，其次介绍了推荐系统的国内外发展情况和旅游系统的发展现状。

第二章介绍了系统的实现过程中应用的主要相关技术，包括前后端分离架构模式、协同过滤推荐算法以及 Mahout 推荐系统架构。

第三章介绍了系统的需求方案，详细梳理了普通用户端和管理员用户端的具体功能需求，同时提出了身份权限控制、界面建议美观、系统可扩展性等非功能性需求。

第四章介绍了系统的设计方案，对系统进行整体模块设计，划分为信息收集和整合模块、推荐模块、信息保存及展示模块三大模块，详细介绍了各模块的具体设计，对推荐模块做出重点分析，并设计了系统的功能权限校验机制。详细设计了系统主要七张数据库表中的所有字段结构和它们之间的关系。

第五章介绍了系统的实现，具体阐述了包括功能权限校验机制、推荐功能和信息保存和展示功能的部分代码实现方案，并展示了用户注册及登录页面、系统首页、系统后台管理页面等主要页面的实现结果，给出了推荐算法的评估结果。

第六章总结了全文，对文章工作进行综述。

第二章 系统相关技术

系统基于 Java 语言编写，设计与实现使用了以下主要技术：前后端分离架构模式、协同过滤推荐算法、Mahout 推荐系统架构。下面将依次介绍主要技术与其在系统中的作用情况。

第一节 相关技术介绍

2.1.1 前后端分离架构模式

过去的公司后端开发者往往还需兼顾前端工作，同时实现页面开发和 API 接口开发，开发者压力大且效率低，系统的耦合程度高，可维护性和可扩展性极差。因此产生了前后端分离的架构模式，通过 Tomcat+Nginx 有效进行解耦，如今已成为互联网企业级开发的标杆。前后端分离开发，解决了前后端开发者分工不均的问题，使后端开发者能专注具体业务逻辑的开发，前端开发人员则关注页面样式，处理更多动态数据的解析和渲染，两者不相互依赖，都可以独立开工，大大提高了开发效率。

本系统的后端使用基于 Java 语言的 Spring Boot 框架开发，前端使用基于 JavaScript 语言的 Vue.js 框架开发，以下将对两种技术进行介绍。

Spring Boot 框架

Spring Boot 是由 Pivotal Team 设计的一种开箱即用的框架，它通过使用了各种默认设置，可以大大简化项目配置，让 Spring 的应用程序更加轻量，开发人员也能够更快入门。通过在主程序中执行 main 函数即可运行 Spring Boot 应用程序，开发者还可以将应用程序打包为 jar，并使用 Java jar 运行 web 应用程序。Spring Boot 遵循“约定优先于配置”的原则，使用它只需要最少量的配置，大多数情况下，可以直接使用默认配置。Spring Boot 框架被广泛应用于开发基于 Spring 框架的应用程序或 Restful 服务，相比于 Spring 框架更聚焦于纯粹地定义应用程序的类型或特征，Spring Boot 框架使开发者更专注于具体的业务功能开发，隐藏了复杂的配置信息。

Spring Boot 框架遵守着一个分层体系结构，其中每一层都与直接上（下）层进行通信，四个层级如 2.1 所示。

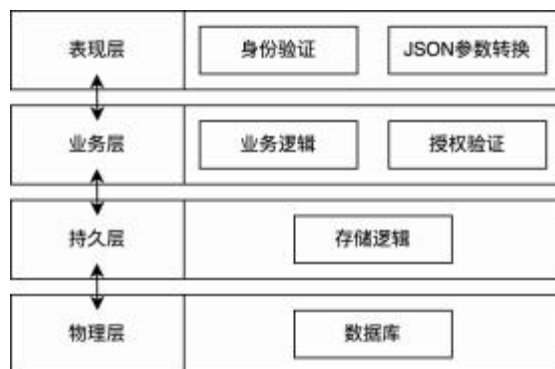


图 2.1 Spring Boot 架构图

表示层处理前端发送的 Http 请求，将 JSON 参数和对象实体相互转换，对请求进行身份验证后与业务层交互。业务层完成所有业务逻辑的处理，由多个服务类构成，调用持久层提供的数据库访问服务，并且完成一些授权和验证工作。持久层完成所有存储逻辑，业务对象和数据库的实体对象在此进行转换。物理层即为真实数据库，存储全量数据信息，在此完成真正的数据库操作（创建、查询、更新、修改、删除等）。

Vue 框架

Vue 框架是一个用于构建用户界面的渐进式 JavaScript 框架。与其他大型框架不同，Vue 被设计为自下而上、逐层应用、增量开发。Vue 的核心库只专注于视图层，不仅易于入门，而且便于与第三方库或现有项目集成。但是 Vue 不仅仅是一个简单的视图库，它还使用一系列周边工具组件使构建大型应用程序变得易学易用，成为目前开发者喜爱的主流前端框架。

Vue 有两个核心功能：

1. 声明性渲染：Vue 扩展了基于标准 HTML 的模板语法，允许声明性地描述最终输出的 HTML 和 JavaScript 状态之间的关系。
2. 响应性：Vue 自动跟踪 JavaScript 状态，并在 DOM 发生变化时以响应的方式更新 DOM。

2.1.2 协同过滤推荐算法

协同过滤算法，从字面理解，即协同操作和过滤操作。协同过滤的思想是通过群体的行为找到一定的相似性（用户或对象之间的相似性），并通过这种相似性为用户做出决策和推荐。所谓协同，即利用群体行为来做出决策（推荐）。生物上存在着协同进化，就是通过协同效应，让群体逐渐进化到更好的状态。对

于推荐系统来说，通过用户的不断协同，最终给用户的推荐会越来越准确。过滤则是从由协同操作推荐后得到的可行决策方案中找到用户偏好的首选方案。

基于协同过滤的推荐算法核心思想是使用 $m \times n$ 矩阵来描述用户对物品的喜爱程度。一般使用评分描述用户对某物品的喜爱程度，分数越高表明用户对该物的偏好越大^[3]。根据评分，形成用户评分矩阵，协同过滤算法过程如2.2所示。

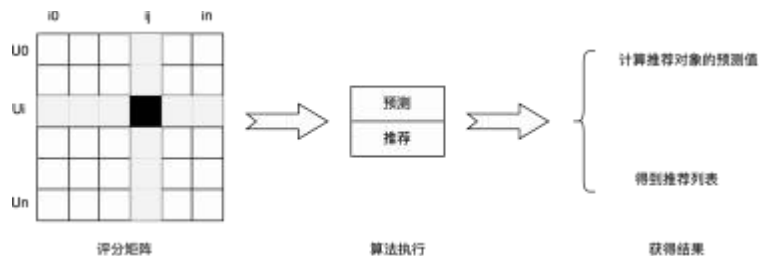


图 2.2 协同过滤过程

本系统中，将用户根据时空信息分组，组内根据评分进行推荐，结构设计更适用于基于用户的协同过滤算法 (User-based Collaborative Filtering, 简称 UBCF)，以下将对该算法进行介绍。

基于用户的协同过滤算法

基于用户的协同过滤算法的本质分为两步：

1. 找出与待推荐用户相似的用户群体。
2. 将用户群体偏好且待推荐用户未选择过的物品推荐给待推荐用户。

实现的主要思路为：统计所有用户对所有景点（可为空）的评分数据，构造评分矩阵等，用于计算用户之间的相似程度，将与待推荐用户最相似的前 K 个用户称为最近邻居^[4]。接着根据统计出的最近邻居用户间互相的共同偏好为待推荐用户做出预测和推荐。假如用户 A 偏好景点 W、Y，用户 C 偏好景点 W、Y、Z，可得出用户 A 与用户 C 相似程度高，考虑将景点 Z 推荐给用户 A，如2.3所示。

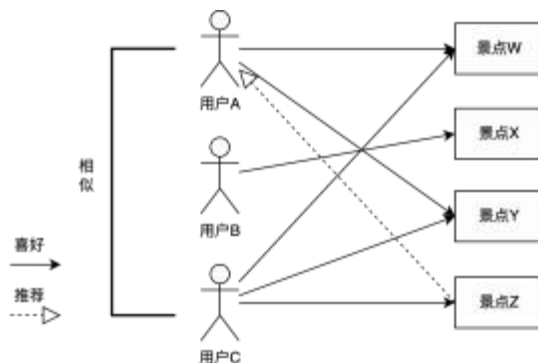


图 2.3 基于用户的协同过滤原理图

基于用户的协同过滤推荐有以下优点：

1. 可以利用其他用户的经验，通过较少的用户反馈即可获取用户感兴趣的内容。
2. 可以基于复杂或难以表述的特征进行过滤。
3. 可以推荐内容上完全不相似的内容，发现用户潜在的喜好，推荐的内容具有新颖性。

2.1.3 Mahout 推荐系统架构

Mahout 是 Apache Software Foundation 旗下的一个开源算法库，它基于 Hadoop 平台编写，提供了机器学习领域经典算法的一些可扩展实现，旨在帮助开发人员更方便、更快速地创建智能应用程序。Mahout 包括许多具体实现，包括聚类、分类、推荐过滤和频繁子项挖掘^[5]。此外，通过使用 Apache Hadoop 库，Mahout 可以有效地扩展到云端。本系统主要使用了 Mahout 框架中基于用户的协同过滤推荐算法。

Hadoop 平台

Hadoop 是一个广受欢迎的开源云计算平台，它作为分布式系统的基础架构，显著特点是高可靠性、高效性、高扩展性以及高容错性，同时成本相对较低。该框架对初学者友好，开发者无需深入探究底层细节，只需基础了解即可利用分布式计算平台开发程序。推荐系统的推荐结果来源于对大量数据的处理与计算，而 Hadoop 正是处理分布式存储与计算的大数据框架。Hadoop 有以下核心组件：

1. HDFS（分布式文件系统）：用于解决海量数据的存储。
2. MapReduce（分布式运算编程框架）：用于解决海量数据的计算。
3. Yarn（作业调度和集群资源管理的框架）：用于解决资源任务的调度。

广义而言，Hadoop 常被理解为一个更为宽泛的概念，即 Hadoop 生态圈^[6]。

Taste 推荐引擎

Mahout 使用名为 Taste 的推荐引擎，Taste 向开发者提供了可扩展接口，基于 Java Interface 实现，开发者可便捷地重写或实现自定义的推荐算法^[7]。

Taste 提供的关键接口有以下几个部分：

1. DataModel Interface：将原始数据映射为 Mahout 兼容的格式，存储了用户偏好等信息。

2. UserSimilarity Interface: 用于计算两个用户间的相似度，类似有 ItemSimilarity 定义了物品间的相似度。
3. UserNeighborhood Interface: 定义了用户之间的“临近”。
4. Recommender Interface: 实现了具体的推荐方法，用于完成推荐功能，包括了训练、预测等部分，是 Taste 引擎的核心。

整体推荐系统架构如2.4所示。

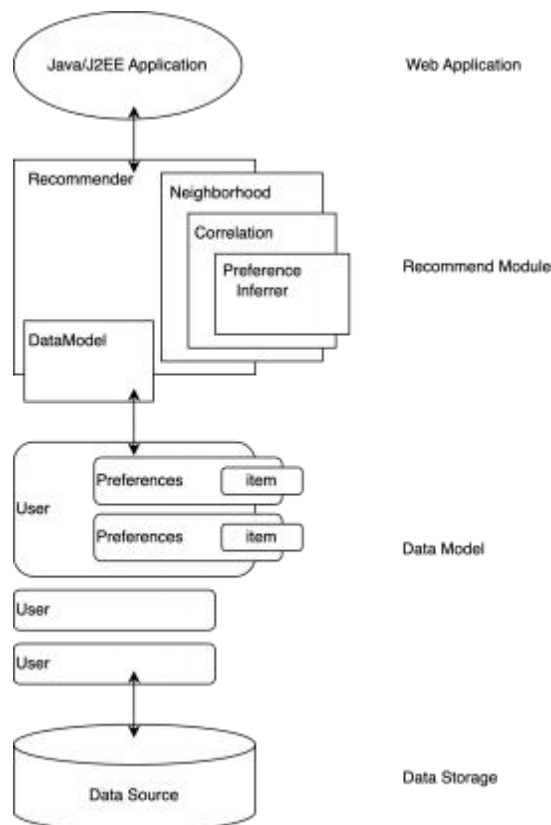


图 2.4 Mahout 推荐系统架构图

第二节 本章小节

本章介绍了系统设计实现中使用到的主要技术，包括前后端分离架构模式、基于用户的协同过滤推荐算法、Mahout 推荐系统架构^[8]。并详细介绍了开发 Web 应用具体使用的技术栈 Spring Boot 框架和 Vue 框架，举例阐述了基于用户的协同过滤算法，最后介绍了 Mahout 依赖的 Hadoop 平台以及 Taste 推荐引擎底层实现使用的抽象接口，此外，对各模块的特性和优点都做出了总结。

第三章 系统需求

本系统是一个基于时空信息向用户推荐合适景点的 Web 应用，旨在协助用户高效、便捷获取感兴趣的景点信息。因此，系统需求分析与用户的实际体验密切相关。本章节做出全面、精确的需求分析，为系统的顺利开发做好基础准备。

第一节 设计目标

本旅游景点推荐系统的主要功能是向游客提供个性化推荐信息，同时也需要系统管理员对系统进行管理和维护，即使用人群有两类：用户（普通游客）和管理员。下面对两类参与者的不同需求进行分析。

3.1.1 用户需求分析

在传统旅游系统中，用户想要获取到感兴趣的景点信息，需要在全部的景点列表中依次查看景点，再根据景点详细信息或用户评价决定其是否为自身感兴趣的内容。这样的决策方式耗时耗力，且有很大可能在大量的浏览后仍然获取不了感兴趣的景点信息。因此本系统的用户端设计最主要的需求是直接向用户推送经推荐算法计算后的景点列表。

首先，用户进入系统首页，未登录状态下默认展示所有景点信息，需要进行信息注册后才可登录，注册时必须填写用户所在位置信息。登录后跳转首页，根据地理位置信息进行推荐计算，并将推荐结果的景点信息向用户展示。景点详细信息可以从首页景点信息，或用户直接搜索关键字查询景点信息进行查看，在详细信息页面，用户可以对感兴趣的景点进行收藏和评分。用户可以对个人收藏的景点列表进行查看和删除。此外，用户可以对个人信息进行修改，主动退出登录。用户需求用例图如3.1。

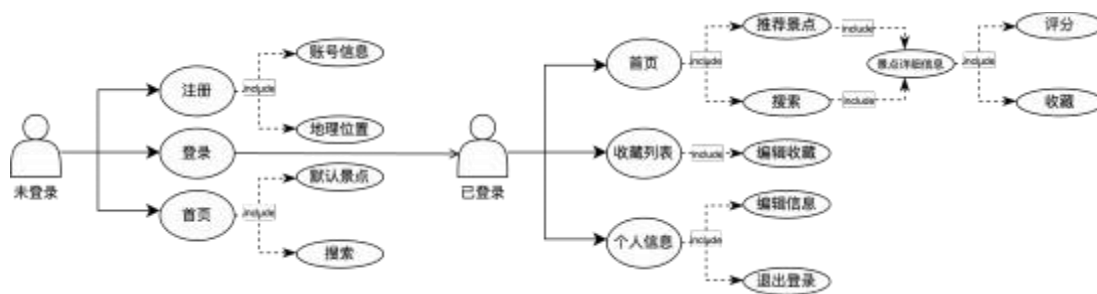


图 3.1 用户需求用例图

根据用户需求分析，系统需要实现的相关功能有：

1. 系统需要实现用户注册、登录以及个人信息编辑功能，注册时加密且持久化保存用户信息，登录时需校验用户输入的用户名、密码，额外添加随机验证码校验。
2. 系统需要实现权限控制和校验功能，区分登录和非登录用户，对收藏、个人信息等登录用户才可查看的页面做权限控制。
3. 系统需要实现推荐算法的自动触发，在登录用户跳转到首页时，根据用户收藏、评分等信息使用基于用户的协同过滤推荐算法进行计算，将结果返回到首页进行展示。
4. 系统需要实现收藏和评分功能，确保用户录入的评分得以保存，用于算法模型的训练。
5. 系统需要实现美观且易用的展示界面，包括景点列表展示、详细信息展示等，降低用户操作难度，提高可用性，并促进客户的使用兴趣。

3.1.2 管理员需求分析

本系统中，管理员角色负责用户信息、景点信息的管理，包括信息查询、增加、删除、修改。此外，管理员可以查看景点收藏量统计信息，为之后的管理决策和系统发展做出参考。

根据管理员需求分析，系统需要实现的相关功能有：

1. 系统需要实现后台权限控制，区分普通用户和管理员，普通用户不得进入管理员后台。
2. 系统需要实现简洁易用的信息管理界面，为管理员提供单条信息的查询、添加、删除、编辑、查看功能，便于管理。
3. 系统需要为管理员提供美观的收藏统计看板，将收藏数量靠前的景点信息制作为统计图进行展示。

第二节 本章小节

本章梳理了系统需求，详细针对用户端和管理员端给出具体的功能需求。除了主要功能，系统也需要关注身份权限控制、界面简易美观、系统可扩展性等非功能性需求。

第四章 系统设计

第一节 系统模块设计

本系统的关键部分为一个推荐系统，通过收集用户的地理位置和对景点的评分信息并存储，经过信息整合，使用推荐算法计算，对用户展示推荐结果。因此可以将主要的推荐功能模块简化为三个部分：信息收集整合模块、协同过滤推荐模块、信息保存及展示模块。划分模块如4.1所示。

其中信息收集和整合模块分为评分数据收集模块和地理位置收集模块，整合后将信息发往推荐模块的 Hadoop 集群中，再使用mahout 推荐框架计算得出推荐结果，将推荐结果传入 Web 应用中的信息保存及展示模块，使信息持久化及美观展示，完成主体推荐功能的数据流转。

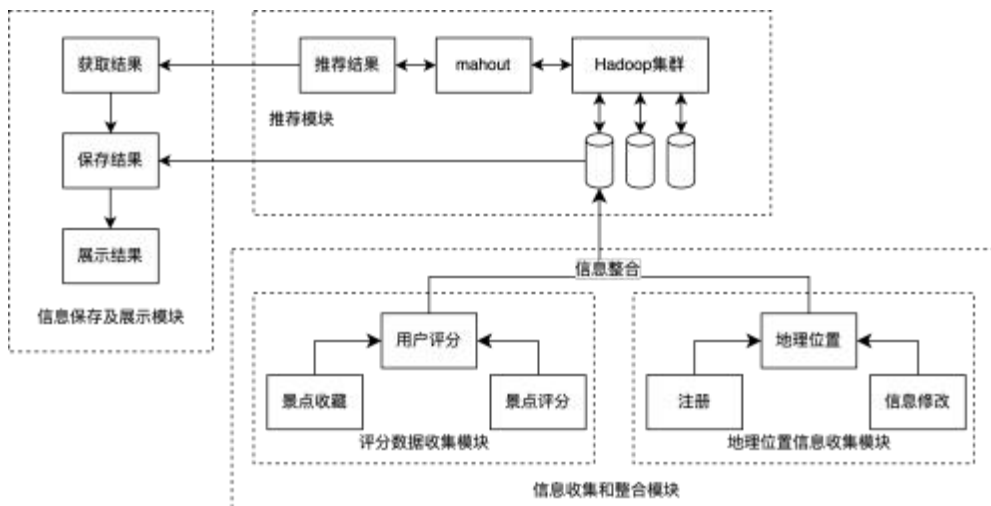


图 4.1 系统模块划分图

以下将对整体模块设计中抽象出的信息收集和整合模块、推荐结果展示模块进行分析设计。推荐模块作为系统重点将在第二节单独进行设计。

4.1.1 信息收集和整合模块

用户的地理位置信息用于将用户分组进行计算，用户的评分信息是保证推荐算法计算准确性的关键，只有信息的收集整理做到完整且可用性高，系统最后的推荐结果呈现才会更加良好，因此信息收集与整合模块非常重要。

用户的地理位置信息由前台注册页面或个人信息修改页面传入，存储在 Mysql 数据库的用户表的字段中。用户的评分信息由前台景点详细信息页面传

入，存储在 Mysql 的景点评分表中。在 Mysql 中，根据用户地理位置名将用户表和评分信息表作聚合，提取用户 (user_id)、景点 (attraction_id)、评分 (score) 三元组，完成信息整合。信息采集及整合的数据流图如4.2所示。

为了避免显式地使用代码统计聚合信息，简化从读取到聚合再到存储的两次 IO 过程，系统使用了数据库的视图特性来对信息做聚合。视图是从一个或多个实体表中导出的表，但与实体表不同，它是一个虚表，只存放定义而不存放真实数据，真实数据仍然在原来的表中。视图类似窗口，只要实体表中的实体表数据发生变化，透过窗口看到的数据也随之变化。使用隐藏了底层表结构的视图提供的统一接口，不仅简化了数据访问，提升了安全性，也简化了系统中使用的聚合提取三元组的操作。

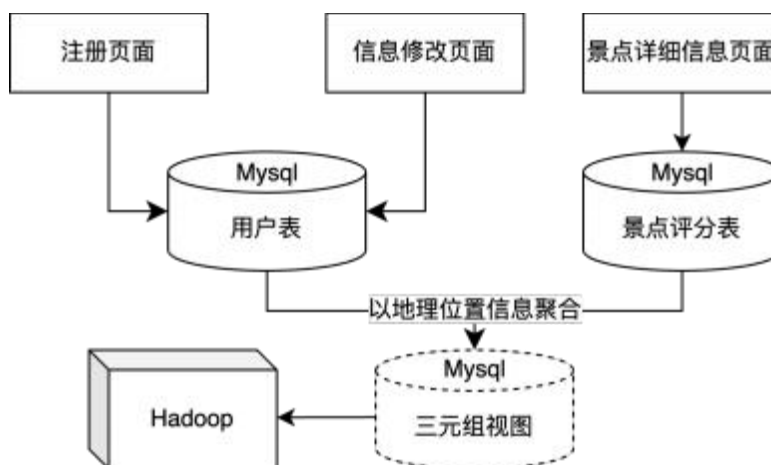


图 4.2 信息采集及整合数据流图

4.1.2 信息保存及展示模块

信息保存及展示模块主要完成系统内多种数据的持久化储存，包括用户账户信息、景点信息、用户评分信息、推荐结果信息等。该模块还要完成各数据从后端到前端的校验和格式转换，使抽象的数据转变为用户可视化的美观页面。主要页面包括：

1. 系统首页推荐结果展示列表，从推荐模块获取推荐结果，从实体对象转换为前台可展示对象。
2. 景点详细信息页面，展示景点信息，并提供收藏选项和评分输入框。
3. 收藏页面，展示用户个人收藏信息。
4. 管理员界面获取收藏数目排名在前的景点列表，并展示为统计图。
5. 管理员界面获取用户信息和景点信息，系统提供增删改查接口，方便管

理员进行信息管理。

4.1.3 功能权限校验机制设计

功能权限校验虽不作为本系统的功能设计重点，但是其对于每个用户生成唯一身份标识，同步用户自身行为数据，并且区分管理员和普通用户可访问页面，防止越权漏洞。功能权限校验机制保证了系统的安全性和可用性，作为系统的非功能性需求的作用至关重要。

本系统用户登录，访问页面时调用后端接口接受信息进行权限认证，使用的是 Springboot 的 SpringSecurity+JWT 方案设计。SpringSecurity 作为核心，JWT 使用签名防止 token 被篡改，保证了 token 的合法性。使用了 JWT 后，系统的认证访问将完全依赖请求中的 Json 对象，服务器则不用保存 Session 数据，即服务器变为更容易扩展的无状态。用户登录及访问其他接口的整体时序图如4.3所示。

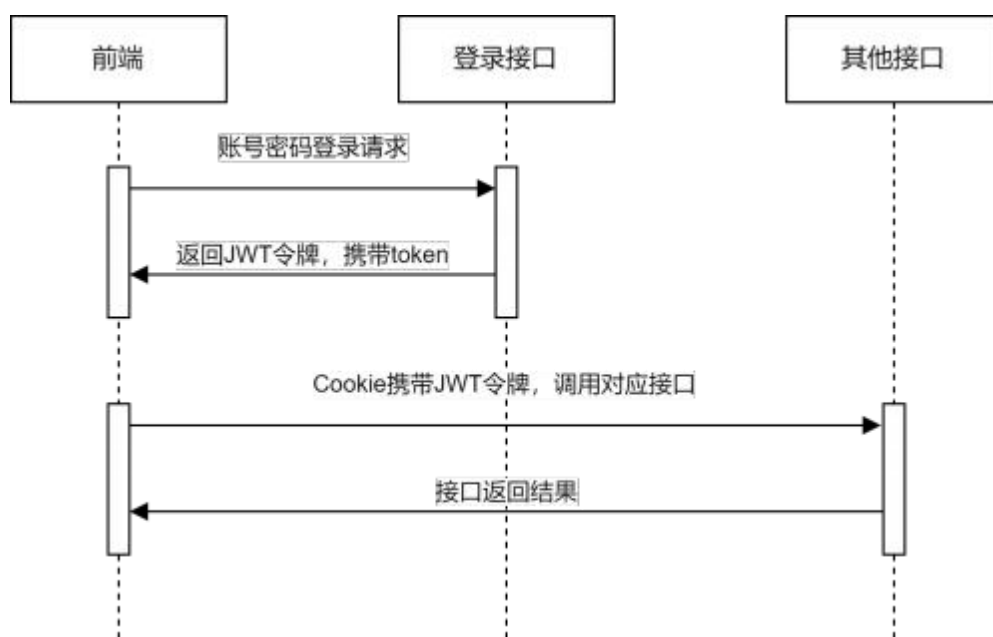


图 4.3 用户权限校验时序图

调用登录接口时，系统完成以下操作：

1. 根据 user_id，到 redis 中获取验证码，对请求携带的验证码做校验。
2. 若验证码无误，则对输入的用户名和密码做校验。
3. 若用户名和密码校验成功，生成一个用户唯一标识的 token。
4. 设置登录时间等登录信息，将登录对象缓存在 Redis 中，并设置过期时间，将一个固定前缀与 token 拼接，设置为存储的 Key。
5. 使用 JWT 对 token 签名生成令牌，返回到前端。

6. 完成登录接口调用后，系统将 JWT 令牌作为 value 存储在 Cookie 中，并以 Admin-Token 作为 Key。

当用户登录后，系统每次请求后端接口时将会被 axios 的请求拦截器拦截，请求头中包含 Authentication Header，这类请求将经过 JwtAuthenticationTokenFilter 过滤器。在过滤器中系统完成以下校验操作：

1. 将 JWT 令牌从请求头 Cookie 中取出，进行 JWT 验签，验签成功则解析出 token。
2. 使用 token 作为 key，从 redis 中取出用户登录数据。
3. 刷新令牌有效期，并把用户登录信息封装为 UsernamePasswordAuthenticationToken 对象，补充到 SpringSecurity 上下文中，用于执行 SpringSecurity 校验。
4. SpringSecurity 校验原理是使用 Controller 接口上的前置校验注解 @PreAuthorize，定义已登录用户才可访问的接口，在执行接口前将会调用鉴权方法，期间会再次从 redis 中获得用户登录信息用于校验，若校验失败则不执行接口方法。

第二节 推荐模块设计

推荐模块是本系统设计的重点部分，需要对数据进行高效的训练和计算。根据系统需求，系统目标是返回接近用户偏好的个性化景点结果列表，因此采用基于用户的协同过滤推荐算法。算法分为三个步骤：

1. 将整合的用户景点评分三元组构建为用户-景点二维矩阵，矩阵值存储评分数据。
2. 计算相似度，统计被推荐用户的前 K 个近邻邻居。
3. 生成最终推荐结果，将预测评分高的前 P 个景点保存并返回，评估推荐效果。

4.2.1 构建用户评分矩阵

在信息采集步骤，用户的收藏和评分行为被纳入统计评分范畴，分值范围设定在 [0, 10] 范围内，有效位数为 1 位。若用户直接给出评分，则取用户评分；若用户作出收藏动作，则设定分值为 6 分；若用户无收藏，但景点地理位置与用户相同，则设定分值为 2 分；其他情况不设置分值。

根据定义规则，接收信息收集和整合模块传入的 (用户 (user), 景点 (attrac-

tion), 评分 (score)) 三元组, 可构建用户-景点二维评分矩阵如公式4.1。

$$M = \langle n \times m \rangle = \begin{bmatrix} S_{11} & S_{12} & \cdots & S_{1j} & \cdots & S_{1m} \\ S_{21} & S_{22} & \cdots & S_{2j} & \cdots & S_{2m} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ S_{i1} & S_{i2} & \cdots & S_{ij} & \cdots & S_{im} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ S_{n1} & S_{n2} & \cdots & S_{nj} & \cdots & S_{nm} \end{bmatrix} \quad (4.1)$$

4.2.2 相似度计算

相似度即两个事物比较后得出的相似性。相似度一般可以使用事物某些特征的距离来表示, 距离小, 则表示事物间相似度高; 距离大, 则表示相似度低^[9]。系统中将对构建的用户评分二维矩阵进行计算, 计算出被推荐用户的前 K 个近邻用户, 以下将对常用的两种相似度进行介绍和比较。

余弦相似度 (Cosine Similarity)

余弦相似度, 即测量 n 维空间内两个 n 维向量的角度余弦值来衡量其相似性^[10]。它等于两个向量的向量积 (点积) 除以两个向量的大小 (长度) 的乘积, 取值范围在 $[-1, 1]$, 取-1 时表示完全不相似, 取 1 时表示完全相似。二维空间中, 向量 $\vec{x}(x_1, y_1)$ 和向量 $\vec{y}(x_2, y_2)$ 的夹角余弦如公式4.2。

$$\cos(\vec{x}, \vec{y}) = \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2} \sqrt{x_2^2 + y_2^2}} \quad (4.2)$$

扩展到 n 维空间, 向量 $\vec{x}(\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n)$ 和向量 $\vec{y}(\vec{y}_1, \vec{y}_2, \dots, \vec{y}_n)$ 的角度余弦相似度如公式4.3, 在 Mahout 框架中调用 `UncenteredCosineSimilarity` 接口计算。

$$UncenteredCosineSimilarity(\vec{x}, \vec{y}) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \times |\vec{y}|} = \frac{\sum_{i=1}^n \sqrt{\vec{x}_i \times \vec{y}_i}}{\sqrt{\sum_{i=1}^n \vec{x}_i^2} \sqrt{\sum_{i=1}^n \vec{y}_i^2}} \quad (4.3)$$

余弦相似度在统计文档的相似度方向有广泛应用, 但在本系统中需要使用用户对景点的评分来计算用户间相似度, 余弦相似度无法将不同用户的评分维度包括在相似度计算中。

皮尔逊相关系数 (Pearson Correlation Coefficient)

Pearson 相关系数^[11] 是在没有维度值的情况下对余弦相似度的改进，将两个向量标准化后再进行计算。计算公式为用协方差除以两个变量的标准差，协方差可以反映两个随机变量之间的相关程度。当协方差大于 0 时，表示两变量之间呈正相关，当协方差小于 0 时，表示两变量之间呈负相关。标准化公式为： $\frac{x-\bar{x}}{\sigma}$ ，对余弦相似度的数据做标准化处理，即可得到皮尔逊相关系数的公式如 4.4。在 Mahout 框架中调用 `PearsonCorrelationSimilarity` 接口计算。

$$PearsonCorrelationSimilarity(\vec{x}, \vec{y}) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (4.4)$$

对评分数据做标准化处理，可以避免用户评分维度差异过大导致余弦相似度计算结果不准确的问题，因此系统最终选择使用皮尔逊相关系数来进行计算。公式 4.1 中的行向量即表示 n 个用户，计算每个 i 用户和 j 用户之间的皮尔逊相关系数，可以得到一个 $n \times n$ 的相似度矩阵 P ，矩阵中 p_{ij} 值表示用户 i 与 j 的相似度。

4.2.3 生成推荐

根据用户相似度计算，可得出与被推荐用户 u 相似度最高的前 K 个近邻用户的集合 $V(u, K)$ 。假设用户 u 感兴趣的景点集合为 $N(u)$ ，将所有 $V(u, K)$ 中感兴趣的景点提取，并去掉用户已收藏的景点得到景点集合 A ，用户初始评分 S_{ui} 乘以相似度矩阵中的 p_{uv} 即可得到对于同一景点 i ，用户 u 和用户 $v (\in V(u, K))$ 的偏好相似程度，公式如 4.5 所示。

$$e'(u, i) = \sum_{\substack{i \in A \\ v \in V(u, K)}} p_{uv} S_{ui} \quad (4.5)$$

在实际推荐中，还需要对用户的评分做标准化处理。例如，一个用户习惯对喜爱的景点打 9 分，对其他的景点打 5 分；另一个用户习惯对喜爱的景点打 10 分，对其他的景点打 6 分。这两个用户的喜好可能非常近似，但直接计算相似程度会被区分。因此最后需要除以用户 u 的相似度和，公式如 4.6 所示。

$$e(u, i) = \bar{S}_u + \frac{\sum_{i \in A, v \in V(u, K)} p_{uv} (S_{ui} - \bar{S}_u)}{\sum_{v \in V(u, K)} p_{uv}} \quad (4.6)$$

将得到的值排序，取相似程度最高的前 P 个景点返回。评估推荐结果使用平均绝对误差（Mean Absolute Error, MAE）和均方根误差（Root Mean Square Error, RMSE）^[12]，范围皆为 $[0, +\infty)$ ，当预测值和真实值完全相同时等于 0，表示模型完美；值越大时表示误差越大^[13]。公式分别如4.7和4.8所示。

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{x}_i - x_i| \quad (4.7)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{x}_i - x_i)^2} \quad (4.8)$$

第三节 数据库设计

本系统的主要数据库表的作用为保存用户、景点基础信息，如用户登录名、密码、地理位置，景点名、景点介绍、地理位置等，由两个实体信息表产生景点收藏、评分信息，保存用户对景点的偏好数据，进而根据地理位置信息分类生成用于推荐运算的三元组视图，最后由推荐模块产生推荐结果信息。系统六张主要的数据表包括用户信息表、景点信息表、景点收藏表、景点评分表、用户-景点分数视图、推荐景点结果表。图4.4为这六张表的 ER 图，以下将对各数据表的具体结构和含义以及它们之间的关系作详细介绍。



图 4.4 系统数据表 ER 图

表 4.1 用户信息表

字段	名称	数据类型	长度	主键	非空	默认值
user_id	用户 ID	BIGINT	20	√	√	
user_name	用户账号	VARCHAR	30		√	
nick_name	用户昵称	VARCHAR	30		√	
user_type	用户类型	VARCHAR	2		√	'00'
phonenumber	手机号码	VARCHAR	11			
avatar	头像地址	VARCHAR	100			
password	密码	VARCHAR	100		√	
login_date	最后登录时间	DATETIME				
create_time	创建时间	DATETIME				
location	地理位置	VARCHAR	255		√	'北京'

4.3.1 用户信息表

用户信息表 (user) 设计如表4.1所示。用户信息表保存了管理员用户和普通用户的基本信息， 由一个自增的 `bigint` 类型 `user_id` 作为主键，唯一标识每个用户。其他重要字段解释如下：

1. 用户账号、昵称、密码、手机号码以及地理位置信息在注册创建用户时填写，并且可以在个人信息设置处修改。
2. 用户密码需经加密后保存， 使用 `Spring Security` 中的 `BCryptPasswordEncoder` 加密算法类完成， 即使明文密码相同， 加密后的密码也不同。
3. 用户类型由长度为 2 的 `varchar` 标识，默认为'00'，即普通用户；超级管理员用户标识为'11'。
4. 地理位置信息为必填选项，用于完成基于位置信息的景点推荐，从包含了我国 34 个省级行政区的字典内选取， 去掉后缀命名， 如：云南（省）、内蒙古（自治区）、北京（直辖市）、香港（特别行政区），默认为'北京'。

4.3.2 景点信息表

景点信息表 (attraction) 设计如表4.2所示。景点信息使用了从去哪儿网爬取的公用数据集，共 3526 个国内景点，经处理后保留景点名、景点地理位置、景点介绍等主要信息。

4.3.3 景点收藏表

景点收藏表 (favorite) 设计如表4.3所示。对于用户和景点，收藏信息都是一对多， 因此单独使用自增的收藏 `id` 作为主键，此表存储所有用户收藏景点的对

表 4.2 景点信息表

字段	名称	数据类型	长度	主键	非空	默认值
address_id	景点主键 id	BIGINT	20	√	√	
name	景点名	VARCHAR	200		√	
location	景点位置	VARCHAR	200		√	
cover	景点主图	VARCHAR	200			
detail	景点介绍	TEXT	65535		√	
create_time	创建时间	DATETIME				

表 4.3 景点收藏表

字段	名称	数据类型	长度	主键	非空	默认值
id	收藏主键 id	VARCHAR	50	√	√	
user_id	用户 id	BIGINT	20		√	
attraction_id	景点 id	BIGINT	20		√	
create_time	创建时间	DATETIME				

应关系。

4.3.4 景点评分表

景点评分表 (mark) 设计如表4.4所示。对于用户和景点，评分信息都是一对多，因此单独使用自增的评分 id 作为主键，此表存储所有用户评分的景点。评分值区间为 [0, 10]，保留一位小数。

4.3.5 用户-景点分数视图表

用户-景点分数视图表由景点评分表与用户信息表根据用户 id 进行左连接（记为结果 tmp1），景点收藏表与用户信息表根据用户 id 进行左连接（记为结果 tmp2），以及用户信息表与景点信息表根据用户 id 进行左连接（记为结果 tmp3）三个结果聚合而成，分别选取其中用户 id(user_id)、景点 id(attraction_id)、分数 (score)。

根据4.2.1中设计，score 在结果 tmp1 中取评分，tmp2 取值 6，tmp3 取值 2，

表 4.4 景点评分表

字段	名称	数据类型	长度	保留小数	主键	非空	默认值
id	评分主键 id	VARCHAR	50		√	√	
user_id	用户 id	BIGINT	20			√	
attraction_id	景点 id	BIGINT	20			√	
score	评分	DOUBLE	8	1 位		√	
create_time	创建时间	DATETIME					

聚合后对整体按 `user_id` 和 `attraction_id` 去重，并按 `score` 排序选取最大值，得到最终用于推荐计算的三元组视图表。该视图表与地理位置的字典一一对应，共计 34 个，以地理位置信息首字母命名，如：云南（yn）、内蒙古（nmg）、北京（bj）、香港（xg）。

以地理位置为‘北京’为例创建视图，使用如下 SQL 语句：

```

1 WITH CombinedResults AS (
2     SELECT user, attraction , score ,
3         ROW_NUMBER() OVER (PARTITION BY user, attraction) AS priority
4     FROM (
5         SELECT u.user_id AS user, m. attraction_id AS attraction , m.score AS score
6         FROM mark m
7         LEFT JOIN user u ON m.user_id = u.user_id
8         WHERE u.location = '北京'
9         UNION ALL
10        SELECT u.user_id AS user, f. attraction_id AS attraction , 6 AS score
11        FROM favorite f
12        LEFT JOIN user u ON f.user_id = u.user_id
13        WHERE u.location = '北京'
14        UNION ALL
15        SELECT u.user_id AS user, a. attraction_id AS attraction , 2 AS score
16        FROM attraction a
17        LEFT JOIN user u ON a.location = u. location
18        WHERE u.location = '北京'
19    ) AS all_results
20 )
21 CREATE VIEW bj AS
22 SELECT user, attraction , score
23 FROM CombinedResults
24 WHERE priority = 1
25 GROUP BY user, attraction;

```

首先使用 WITH 语句创建通用表表达式 (Common Table Expression, CTE)，它定义了一个名为 CombinedResults 的临时结果集，用于暂存三个子查询的合并结果。在 CTE 中使用 ROW_NUMBER() 方法对三条子查询语句根据 user 和 attraction 分组，按行数优先标记优先级 priority。使用 UNION ALL 对子查询结果进行集合合并操作得到 CombinedResults。最后将 CombinedResults 按 user 和 attraction 分组，取 priority=1，即分组后位于第一行的记录，以达到去重的效

果。由于子查询合并的顺序为 tmp1、tmp2、tmp3，因此获取 score 的优先级也为 tmp1>tmp2>tmp3。至此完成用户-景点分数视图表的创建。视图表结构设计如表4.5所示，仅保存 (用户 id(user), 景点 id(attraction), 分数 (score)) 三元组，方便导入推荐模块进行运算。

表 4.5 用户-景点分数视图表

字段	名称	数据类型	长度	保留小数	主键	非空	默认值
user_id	用户 id	BIGINT	20			√	
attraction_id	景点 id	BIGINT	20			√	
score	评分	DOUBLE	8	1 位		√	

4.3.6 推荐景点结果表

推荐景点结果表 (recommend) 设计如表4.6所示。该表用于存储经推荐模块计算后的推荐结果，推荐评分 (recommend_score) 由推荐模块生成。对于被推荐用户，存储按推荐评分从大到小排名的前 16 条记录，作为最终推荐结果，传入结果展示模块，经对象转换处理后最终展示在主页。

表 4.6 推荐景点结果表

字段	名称	数据类型	长度	保留小数	主键	非空	默认值
user_id	用户 id	BIGINT	20			√	
attraction_id	景点 id	BIGINT	20			√	
recommend_score	推荐评分	DOUBLE	8	1 位		√	

第四节 本章小节

本章详细介绍了系统的设计方案，将系统整体划分为信息收集和整合模块、推荐模块、信息保存及展示模块三大模块，分析了各模块内具体实现并详细介绍了重点的推荐模块设计，为推荐模块制定了具体的计算方案，各模块反映了系统整体的架构，同时呈现出系统中主要数据的流动。其次，设计了系统功能权限校验机制，为系统的安全性和可用性提供了保障。

数据库设计中对系统中使用到的用户信息表、景点信息表、景点收藏表、景点评分表、用户-景点分数视图表、推荐景点结果表六张主要数据库表进行了详尽的设计，分析了各数据库表之间的关系，详细设计了各表的结构和所有字段。为系统的最终实现做好准备。

第五章 系统实现

根据系统需求和系统设计的描述，本章节将介绍本系统的各功能和模块的具体实现方式，展示部分代码以及系统呈现效果图。

第一节 系统实现方案

5.1.1 功能权限校验机制的实现

根据4.1.3中对功能权限校验机制的设计，用户进入系统时会进入登录页面，输入用户名和密码后，前端使用 POST 方法向后端发送 Http 请求，后端中实现登录部分的主要逻辑代码如下所示。

```
1  /**
2   * 登录验证
3   *
4   * @param username 用户名
5   * @param password 密码
6   * @param code 验证码
7   * @param uuid 唯一标识
8   * @return 结果
9   */
10 public String login ( String username, String password, String code, String uuid)
11 {
12     // 验证码校验
13     // 较复杂且非重点的用户验证
14     // 生成token
15     return tokenService . createToken ( loginUser ) ;
16 }
```

登录逻辑中，首先对请求携带的验证码作校验，若验证码无误则接着对用户输入的用户名和密码进行校验，最后返回生成的 token。生成token 的逻辑代码 createToken() 方法如下所示。

```
1  /**
2   * 创建令牌
3   *
4   * @param loginUser 用户信息
5   * @return 令牌
```

```
6  */
7  public String createToken(LoginUser loginUser)
8  {
9      String token = IdUtils.fastUUID();
10     loginUser.setToken(token);
11     // 保存其他登录信息
12     setUserAgent(loginUser);
13     // 设置缓存
14     refreshToken(loginUser);
15
16     Map<String, Object> claims = new HashMap<>();
17     claims.put(Constants.LOGIN_USER_KEY, token);
18     return createToken(claims);
19 }
20 /**
21  * 从数据声明生成令牌
22  *
23  * @param claims 数据声明
24  * @return 令牌
25  */
26 private String createToken(Map<String, Object> claims)
27 {
28     String token = Jwts.builder()
29         .setClaims(claims)
30         .signWith(SignatureAlgorithm.HS512, secret).compact();
31     return token;
32 }
```

使用 `IdUtils.fastUUID()` 方法生成唯一标识用户的 `token`，将其和其他登录信息，如登录 `ip`、地址、浏览器等，一起保存到 `LoginUser` 对象中，同时刷新 `Redis` 缓存设置，设置当前登录时间和过期时间。接着将 `token` 包装到 `Map` 里，用于保存到 `Jwts` 对象中，再调用 `Jwts.signWith()` 方法签名生成最终的 `JWT` 令牌，最终返回到前端。

登录后用户每次的访问产生的系统请求，将经过 `UsernamePasswordAuthenticationFilter` 过滤器进行 `JWT` 验证签名，验签完成则解析出 `token`，继续经过继承了 `OncePerRequestFilter` 的 `JwtAuthenticationTokenFilter` 过滤器进行 `JWT` 令牌校验，重写的 `doFilterInternal()` 方法如下所示。

```

1 @Override
2 protected void doFilterInternal ( HttpServletRequest request , HttpServletResponse response ,
    FilterChain chain )
3     throws ServletException , IOException
4 {
5     // 获取LoginUser对象
6     LoginUser loginUser = tokenService . getLoginUser( request ) ;
7     if ( StringUtils . isNotNull( loginUser ) && StringUtils . isNull ( SecurityUtils .
    getAuthentication () ) )
8     {
9         // 检查token是否过期，并刷新token
10        tokenService . verifyToken( loginUser ) ;
11        // 封装用户登录信息
12        UsernamePasswordAuthenticationToken authenticationToken = new
    UsernamePasswordAuthenticationToken(loginUser, null , loginUser . getAuthorities () ) ;
13        authenticationToken . setDetails ( new WebAuthenticationDetailsSource () . buildDetails (
    request ) ) ;
14        // 保存到SpringSecurity上下文
15        SecurityContextHolder . getContext () . setAuthentication ( authenticationToken ) ;
16    }
17    // 继续后续过滤逻辑
18    chain . doFilter ( request , response ) ;
19 }

```

首先根据 token 从 Redis 中获取出 LoginUser 用户登录信息对象，接着验证 token 是否过期，并刷新 token 过期时间，最后封装用户的登录信息保存到 SpringSecurity 上下文中，继续后续过滤逻辑，这样保证了只有登录用户才可以访问部分功能，并且用户长时间不操作将会释放缓存，重新访问时需要再次登录。SpringSecurity 会对加了前置校验注解 @PreAuthorize 的借口进行鉴权，以此来保证不会发生越权漏洞。

5.1.2 推荐功能的实现

系统使用 Mahout 推荐算法框架完成基于用户的协同过滤推荐算法，算法实现的伪代码如算法1所示。

Mahout 提供了 MySQLJDBCDataModel 接口，可以从 Mysql 数据库中直接读取用户-景点分数视图表中的数据，使用 DataModel 接口保存用户对景点的

算法 1 推荐算法实现过程**输入：**被推荐用户 (user)**输出：**推荐景点结果集合 (recommendations)

```

1: dbTable = getDbTable(userId) // 根据地理位置获取用户所在的景点分数视图表名 (如 bj)
2: dataModel = new MySQLJDBCDataModel(dbTable, "user_id", "attraction_id", "score")
  // 使用DataModel 接口保存用户-景点评分数据
3: for each otherUser  $\in$  userSet do
4:   if user  $\neq$  otherUser then
5:     similarity = PearsonCorrelationSimilarity(user, otherUser) // 使用皮尔逊相似度计算
6:     if similarity > 0 then
7:       ratingList.add(similarity, otherUser)
8:     end if
9:   end if
10: end for
11: neighbors = new NearestNUserNeighborhood(ratingList, dataModel, K) // 对 ratingList 排序, 取前 K 个相似用户为近邻邻居
12: recommender = new GenericUserBasedRecommender(dataModel, ratingList, neighbors) // 构建推荐器
13: recommendations = recommender.recommend(user, P) // 进行推荐, 返回分数最高的前 P 个推荐景点=0

```

评分信息^[14]。接着可对 dataModel 中的用户集合做与被推荐用户之间的相似度计算, 根据4.2.2中的设计, 使用 PearsonCorrelationSimilarity 接口计算皮尔逊相似度, 使用 UserSimilarity 保存对象, 表示用户之间的相似度。Mahout 框架获取前 K 个相似用户作为近邻邻居, 使用 NearestNUserNeighborhood 对象构建, 传入计算得到的相似度、dataModel 以及指定数量 K 即可获得固定数量的邻居。最后使用 GenericUserBasedRecommender 对象构建推荐器, 将被推荐用户和指定数量 P 传入 recommend 方法即可获得推荐分数最高的前 P 个推荐景点。

此外, Mahout 提供了推荐系统的评估方法, 系统使用基于 Prediction-based 方法的 MAE 和 RMSE 指标对推荐结果进行评估, 调用框架内的 AverageAbsoluteDifferenceRecommenderEvaluator 评估器, 评估部分代码如下。

```

1 // 参数0.7表示评估的训练集为70%, 1.0代表所有的用户来参与评估
2 RecommenderEvaluator evaluator = new AverageAbsoluteDifferenceRecommenderEvaluator();
3 RecommenderBuilder recommenderBuilder = dataModel -> recommender;
4 double score = evaluator.evaluate(recommenderBuilder, null, dataModel, 0.7, 1.0);
5 double rmse = recommenderEvaluator.evaluate(recommenderBuilder, null, dataModel, 0.7, 1.0);
6 System.out.print("指标评估: score: " + score + " rmse: " + rmse + "\n");

```

5.1.3 信息保存和展示功能的实现

根据4.1.2中的设计，信息保存和展示模块需要对多种数据进行持久化储存，并需要对推荐结果等数据进行美观展示。后端在这部分主要进行对象转换，使用 Mybatis 与数据库进行交互，使用大量 SQL 语句完成数据的增删改查操作。前端使用 Vue.js 框架完成页面组件的整合和路由，呈现出美观页面。以下展示推荐结果经后端对象转换展示到前端的部分代码。

从推荐算法得到的 `RecommendedItem` 对象仅保存了景点的id，首先需要经过数据库查询得到 `Attraction` 对象，对用户的收藏过滤后，再使用 `getDataTable()` 方法统一转换为前端可展示的 `TableDataInfo` 对象。

```

1  @GetMapping("/list")
2  public TableDataInfo list () {
3      // 推荐参数前置处理，获取用户登录信息等
4      ...
5      // 获取推荐结果，此时数据为RecommendedItem对象
6      List<RecommendedItem> recommendedItems = RecommendUtils.recommendUserCF(userId,
7      100, initials);
8      if(recommendedItems != null)
9          for (RecommendedItem recommendedItem : recommendedItems)
10             idList .add(recommendedItem.getItemID());
11     for(Long id: idList ) {
12         // 将RecommendedItem对象转换为Attraction对象
13         Attraction attraction = attractionService . selectAttractionById ( String .valueOf(id)) ;
14         if( Objects .nonNull( attraction ))
15             if( list .size () <= 16)
16                 // 若当前用户没有收藏该景点，则将景点保存，此处即为协同过滤的过滤方法
17                 if(! list .contains ( attraction )) {
18                     if(! attraction .getCover().contains (" localhost :8080"))
19                         attraction .setCover("http :// localhost :8080" + attraction .getCover())
20                     ;
21                     list .add( attraction );
22                 }
23     }
24     // 将包含Attraction对象的List转换为前端可展示的TableDataInfo对象，返回到前端
25     return getDataTable( list );
26 }

```

第二节 系统实现结果

5.2.1 用户注册及登录页面

注册和登录页面展示如5.1和5.2所示。用户进入系统后，首先会直接进入登录页面，若已有账号直接登录即可；若无账号，可在页面上方选择注册选项，进入注册页面，其中用户名、密码、地理位置信息为必填，若未填写系统会报错，注册无法成功。



图 5.1 用户登录界面效果图



图 5.2 用户注册界面效果图

5.2.2 系统首页

系统首页主要展示经推荐算法计算后得到的推荐景点，用户可以点击景点进入详细页面查看，也可以在此对感兴趣的景点进行搜索，页面展示如5.3和5.4所示。



图 5.3 首页顶部及搜索效果图



图 5.4 首页推荐景点效果图

在系统运行后台，会在终端显示推荐过程中计算的近邻用户、结果列表和对应评分，前端展示的景点列表与计算结果的景点列表一一对应。同时系统对推荐结果进行评估，打印出 MAE 和 RMSE 指标。后台显示推荐结果和评估结果如5.5和5.6所示。



图 5.5 推荐结果图



图 5.6 评估结果图

评估系统推荐情况的 MAE 值为 1.2 左右，RMSE 为0.94 左右，在用户数据量为千内量级时，可以认定系统的推荐效果满足需求，系统具备可用性。

5.2.3 系统后台管理页面

系统后台管理页面如5.7所示，在此管理员可以对景点数据进行增加、删除、查询，对景点的主图、地理位置、详情等详细信息进行上传和编辑。



图 5.7 后台管理页面图

第三节 本章小节

本章展示了系统的具体实施方案以及系统的呈现结果。

在系统实施方案中，详细介绍了功能权限校验机制、推荐功能以及部分信息保存和展示三大重要功能的实现。功能权限校验机制使用 **SpringSecurity+JWT** 令牌实现，在登录是创建 **JWT** 令牌，并在之后系统的每次请求中携带并校验，为系统提供安全性保证。推荐功能使用 **Mahout** 框架，可以便捷调用相关接口，保存用户-景点分数表的数据，计算与被推荐用户的相似度，筛选出前 **K** 个近邻居，最后使用推荐器推荐出分数最高的前 **P** 个景点，还可以调用框架内的评估器对推荐结果进行评估，完成系统的重点推荐功能。信息保存和展示功能展示了推荐结果从后端到前端的对象转换方案，使数据能正确在前后端分离架构中流动。

系统实现结果中，展示了用户注册及登录、系统首页、系统后台管理页面以及推荐结果数据和评估效果。美观简易的界面，表明系统具有可用性。评估结果的 **MAE** 和 **RMSE** 指标较好，表明系统实现的推荐算法作为核心具有正确性。

第六章 文章总结

在如今，旅游业已成为国家经济和人们娱乐方式中不可或缺的重要产业之一。传统旅游景点系统为用户提供了便捷查询景点信息和其他用户评价的平台，却往往忽略了用户的时空信息、收藏和评分等个性化偏好。文章因此设计并实现了基于时空信息的旅游景点推荐系统，系统将根据用户的地理位置、收藏景点历史和景点评分，主动向用户推送用户可能偏好但自身并未收藏的景点。使用本系统，用户将不用在海量景点信息中消耗时间与精力却难以发掘自身喜好的景点，系统提供了美观易操作的可视化界面，并制定了安全可靠的权限校验方案，在旅游系统产业中具有价值发挥空间。

文章详细介绍了系统使用的相关技术、系统的用户及管理员需求、系统的具体模块设计、算法设计及数据库设计，并展示了实际的实现方案和实现效果。按照文章所述完成的系统，最终具备所有预期功能，推荐算法评估结果也达到预期，可以为用户提供基于时空信息的景点推荐。

参考文献

- [1] Resnick, Varian. Recommendation Systems [J]. Communications of the ACM, 1997, 40(3): 56–58.
- [2] Burgess S, Sellitto C, Cox C, *et al.* Trust perceptions of online travel information by different content creators:some social and legal implications [J]. Service Industries Journal, 2009, 15(2): 162–168.
- [3] 张磊. 基于 Mahout 的高校图书馆藏书推荐系统的设计与实现 [mathesis], 2022.
- [4] Yang Deguo, Shi Qing. Implementation of Personalized Recommendation Algorithm based on Mahout [J]. Journal of Physics: Conference Series, 2021, 1827(1).
- [5] 肖锦波. 基于 MapReduce 的医学数据并行聚类算法研究 [mathesis], 2016.
- [6] 李正洋. 基于大数据的用户画像系统的设计 [mathesis], 2022.
- [7] 吴彦文, 齐旻, 杨锐. 一种基于改进型协同过滤算法的新闻推荐系统 [J]. 计算机工程与科学, 2019, 1187(5): 1179–1185.
- [8] 刘金羽. 前后端分离的在线考试系统设计与实现. 电脑编程技巧与维护, 2020: 44–46.
- [9] 韩宁. 基于 POI 和 Dijkstra 算法的移动机器人调度系统设计与实现 [D]. 湖南大学, 2018.
- [10] 李朝东. 改进的宽动态范围图像的增强算法. 云南民族大学学报 (自然科学版): 1–10.
- [11] Berthold, M.R., Höppner. On clustering time series using euclidean distance and pearson correlation [F]. arXiv preprint arXiv:1601.02213, 2016.
- [12] 白伟, 李凤英. 基于 BP 神经网络的马铃薯价格预测. 价值工程, 2020, 39: 201–203.
- [13] 张旺, 管维亚, 张建峰, *et al.* 基于 XGBoost 算法的基础设施项目投资预测模型——以变电站工程为例. 土木工程与管理学报, 2021, 38: 78–84.
- [14] ZHANG Jiani, SHI Xingjian, ZHAO Shenglin, *et al.* STAR-GCN: Stacked and reconstructed graph convolutional net works for recom-mender systems [J]. Information Retrieval, 2019: arXiv:1905.13129.

致 谢